

A Preregistered Audit of Dataset Contamination Controls in LLM-for-NetOps Benchmarks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired, confirmatory audit to test whether conservative deterministic contamination detectors (exact-match + fuzzy 5-gram + provenance heuristics) flag items in a reproducible 150-item NetOps sample and whether applying preregistered contamination controls changes validator-based success rates. Crucially, the executed sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") and shared generation semantics (independent_condition_generation = false), recorded in the archived model and task manifests; these two execution facts materially limited the audit's sensitivity to generation-level contamination. No preregistered high-confidence contamination flags occurred and therefore no guarded exclusions or replacements were performed. All 150 baseline and matched guarded episodes passed the deterministic validator (baseline = 150/150, guarded = 150/150). The paired success-rate difference = 0.0 percentage points (95% CI [0.0, 0.0]); exact McNemar two-sided $p = 1.0$. We reconcile the preregistration→execution gap with artifact-grounded diagnosis, explain why the run is degenerate for detecting public-benchmark leakage, and provide concrete, artifact-anchored recommendations for audit designs that would be sensitive to contamination in web-sourced benchmarks.

I. INTRODUCTION

Motivation. Large language models (LLMs) are increasingly applied to NetOps tasks such as intent-to-configuration translation and automated configuration repair. Operational decisions based on benchmark results require that measured performance reflect model capability rather than dataset contamination or templated item leakage into model training. Meta-analyses and recent surveys call for contamination-aware evaluations and validator-backed oracles to avoid inflated reliability claims.

Research question and contribution. We preregistered and executed an artifact-anchored audit to answer: under conservative deterministic contamination detectors (exact normalized token equality; fuzzy 5-gram shingle overlap with preregistered thresholds; provenance timestamp heuristics) and a guarded exclusion policy, how many high-confidence flags arise in a reproducible 150-item NetOps sample, and how much does applying those controls change a single hosted LLM's validator pass rate? Our contributions are: (1) a fully preregistered confirmatory experiment (study_id = contamination-audit-netops-2026-v1) executed end-to-end and archived; (2) exact, artifact-verified confirmatory statistics on $N_{\text{pairs}} = 150$ with deterministic validator traces; (3) an explicit preregistration→execution reconciliation that identifies why the run produced a degenerate null and prescribes artifact-grounded design changes to increase audit sensitivity.

Scope and bounded claims. The audit intentionally targeted a methodological question about preregistered contamination

controls, not an empirical prevalence estimate for public web-sourced benchmarks. The executed confirmatory sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") produced by the deterministic task generator and employed shared_initial_candidate generation semantics (independent_condition_generation = false). Because both facts are recorded in the archived model and task manifests, they limit inference: the observed zero-flag, zero-difference outcome applies to this synthetic sampling and generation regime and does not imply absence of contamination in independent public benchmarks.

II. RELATED WORK

LLM-driven NetOps systems and validators. Prior system and benchmark papers demonstrate that LLMs can translate intent into configuration artifacts and can propose repairs; integrating verifiers or iterative validate–repair loops improves average repair quality compared to naive generation. Formal network verification tools (e.g., Header Space Analysis and localized subspecification) are practical oracles for reachability and policy correctness and are commonly used in generate–validate pipelines.

Evaluation-validity critiques and contamination audits. Surveys and meta-analyses highlight two recurring evaluation risks: (a) contamination—benchmark items or templated prompts appearing in model training data can inflate measured accuracy; and (b) construct invalidity—benchmarks dominated by low-complexity or templated tasks may not represent operational difficulty. Recommendations in the literature include deterministic fingerprinting, fuzzy n-gram overlap, provenance tracing, and reporting of detector score distributions. Our audit implements a conservative subset of these defenses and emphasizes an artifact-anchored preregistration→execution reconciliation.

Positioning. Unlike broad capability benchmarks that measure absolute performance across model families, this study evaluates whether a specific preregistered set of contamination controls would alter validator-measured success on a reproducible sample under the archived execution semantics. Where prior works focus on detector design or end-to-end agentic repair effectiveness, our contribution is a small, fully preregistered audit and a tight artifact-grounded diagnosis of why the experimental choices produced a degenerate result.

III. METHODOLOGY

Preregistration and primary estimand (design freeze). The study was preregistered under study_id = contamination-audit-netops-2026-v1 and froze the confirmatory design (estimands, sampling, detectors, exclusion/replacement rules, and analysis procedures) prior to observing model outputs. The primary estimand is the paired success-rate difference (guarded - baseline) across item×model pairs; the preregistered confirmatory test is an exact McNemar two-sided test and paired bootstrap percentile 95% CI computed with 10,000 resamples and seed = 1729. The registered analysis executor is paired_binary_analysis_v1 and the preregistration explicitly specified sensitivity runs (alternate fuzzy thresholds and failure-as-zero handling) and a multiplicity plan that treats the primary test as the single confirmatory inference.

Contamination detectors and guarded policy (operationalized). The audit implemented three deterministic detectors as preregistered defenses: (1) exact normalized token equality after canonicalization (whitespace and token normalization); (2) fuzzy 5-gram shingle overlap (Jaccard/overlap) computed on 5-gram token shingles with preregistered high-confidence threshold ≥ 0.80 and medium band $[0.50, 0.80)$; and (3) provenance timestamp heuristics that compare item source timestamps/DOIs against the model public-cutoff metadata. The detection pipeline was implemented to produce high-confidence flags when exact equality OR 5-gram overlap ≥ 0.80 hold; medium flags were marked for overlap in $[0.50, 0.80)$ or provenance risk. The preregistered guarded policy deterministically excludes only high-confidence flagged items and replaces them with deterministic retemplated + topology-augmented items sampled from the same generator using fixed replacement seeds; replacements were required to satisfy an embedding-similarity constraint (preregistered cosine threshold ≥ 0.85) and to pass functional sanity checks before being accepted. The detector implementation persisted flagged events and replacement traces by design; the code path for detailed per-item overlap table persistence was activated only when threshold crossings or replacements occurred (see Results for the artifact consequence).

Sampling, task generation, and generation semantics (preregistered options). The preregistration allowed sampling from a specified benchmark scope; for reproducibility, the design permitted using the deterministic task generator (deterministic_netops_task_generator_v5) with fixed generation seeds and specified source identifiers. The preregistered execution_contract allowed independent_condition_generation (two independent model calls per item, producing two independent outputs) or shared_initial_candidate semantics (single shared generation reused across conditions) depending on experimental constraints. The confirmatory sample target was task_count = 150 items (planned_episode_count = 300 episodes if independent generations were used). The sampling plan included deterministic replacement rules for excluded items and explicit seeds for replacement sampling so that paired alignment across baseline and guarded conditions is preserved in the event of

exclusions.

Model and provider configuration (declared). The execution contract specified a single hosted model family for the confirmatory run (declared_model_version = "gpt-5-mini"). The model configuration used in the pipeline (documented in execution/model_configuration.json in the archive) set provider = openai_responses, max_output_tokens = 2000, maximum_attempts_per_call = 1, reasoning_effort = "minimal", and store = false. The preregistration capped maximum_model_calls = 300 (one per condition per item under independent generation), but allowed shared-generation runs to consume fewer calls when allowed by the design.

Deterministic NetOps verifier and scoring rule (oracle). The verification oracle is deterministic_netops_validator_v5 (version 5.0). The validator follows a parse $\rightarrow$ state-update $\rightarrow$ property-check pipeline: it parses candidate configuration commands into normalized command sequences, simulates the command sequence against the canonical initial state to produce an expected final state, and evaluates a set of preregistered workflow and invariant constraints (e.g., required command counts, ordered shutdown/restore patterns, protected interfaces, group availability constraints). The preregistered scoring rule is full_intent_constraint_validation: a binary pass requires reaching the intended final state while respecting all workflow constraints; the validator also records diagnostic fields per episode (normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_codes) and archives detailed traces for reproducibility. These trace fields form the deterministic basis for the paired binary outcomes used by the paired analysis executor.

Missingness, retries, and analysis handling (preregistered). The preregistered missingness plan mandated a single automated retry for model/API call failures within a five-minute window; if a retry failed, the analysis default was 'complete_pair_primary' (the pair treated per the analysis executor rules) with a sensitivity run treating failed calls as failures (zero). Exclusion rules were deterministic and applied only to high-confidence flagged items; replacements were deterministic and seeded. The preregistration included sensitivity analyses for fuzzy thresholds at 0.50 and 0.65 and subgroup analyses by templating detector output.

Planned power and interpretation guidance (design constraints). The preregistration documented power guidance acknowledging that a two-generation per item design (independent_condition_generation = true) or a multi-model plan materially increases the number of independent trials and therefore the power to detect modest paired effects (target detect ≈ 5 percentage points with 80% power under conservative assumptions). In contrast, shared_generation semantics (single shared model call per item) reduce the number of independent model calls and therefore lower sensitivity to generation-level contamination unless deterministic high-confidence flags occur. These planning notes were preregistered to guide interpretation of null or degenerate paired outcomes.

Reproducibility, archival, and artifact paths. All sampling

seeds, generator identifiers, validator implementations, detector code, and analysis executors were recorded and archived as part of the execution bundle. Key artifact paths produced by the pipeline include `execution/task_manifest.jsonl`, `execution/model_configuration.json`, `execution/raw_results.jsonl`, `analysis/paired_contingency_table.csv`, and `analysis/contamination_summary.csv`; the full execution manifest contains hashes and a complete artifact index. The preregistration SHA and the manifest entries are archived and available for verification in the execution bundle. The analysis executor parameters (bootstrap resamples = 10000, bootstrap_seed = 1729) and the exact paired_test method (exact_mcnemar_two_sided) were recorded in the analysis log and are reproduced in the archived analysis artifacts.

IV. RESULTS

Execution accounting and deterministic reconciliation. The archived execution manifest records `planned_episode_count` = 300 (150 items $\times$ 2 conditions) and `completed_episode_count` = 300 with `failed_episode_count` = 0. Because `independent_condition_generation` = false and no high-confidence exclusions occurred, the pipeline used `model_calls_used` = 150 (one shared generation per item) rather than two separate generations per condition; provider-call accounting and reconciliation artifacts confirm these counts and semantics (see `execution/model_configuration.json`, `execution/task_manifest.jsonl`, and `analysis/deterministic_reconciliation.json`). Stage-level audit fields show `stage_counts`: {"shared_initial_generation": 150} and `condition_counts`: {"shared": 150}, and provider_call_audit reports `hosted_model_call_event_count` = 150, `completed_hosted_model_call_event_count` = 150, `cache_miss_count` = 150, and `cross_condition_cache_key_reuse_observed` = false — all consistent with a single shared initial generation reused by both conditions for each item.

Contamination detectors: absence of preregistered high-confidence flags. The preregistered detectors produced zero high-confidence contamination flags for the executed sample; the analysis artifact `analysis/contamination_summary.csv` contains no flag rows. The detector workflow was implemented to persist per-item fuzzy overlap scores only when threshold crossings or replacements were required; because no items met the preregistered high-confidence threshold (exact OR 5-gram ≥ 0.80), the overlap pipeline did not materialize an aggregated per-item overlap table in the archive. This empty flag summary is an explicit artifact result (`analysis/contamination_summary.csv`) and not an absence inferred from silence: the detector logged no threshold events to persist. Consequently, no guarded exclusions or deterministic replacements were performed (`replacements_performed` = 0 in the execution manifest).

Validator outcomes and confirmatory statistics. The `deterministic_netops_validator_v5` returned pass for all baseline episodes (`baseline_success_count` = 150/150;

`baseline_success_rate` = 1.0) and for all guarded episodes (`guarded_success_count` = 150/150; `guarded_success_rate` = 1.0). The paired contingency table archived as `analysis/paired_contingency_table.csv` is `n11` = 150, `n10` = 0, `n01` = 0, `n00` = 0. The primary estimand (`paired_success_rate_difference_guarded_minus_baseline`) = 0.0 percentage points. The preregistered paired bootstrap percentile 95% CI (10,000 resamples, bootstrap_seed = 1729) is [0.0, 0.0]; the exact McNemar two-sided test yields `p` = 1.0. These numbers are computed by the archived `paired_binary_analysis_v1` executor and are recorded in `analysis/results.json` and `analysis/condition_summary.csv` (`condition,successes,failures,observed,success_rate: baseline,150,0,150,1.0; guarded,150,0,150,1.0`).

Representative episode diagnostics and difficulty coverage. Representative archived episodes span the preregistered difficulty levels (medium, hard, very_hard) and illustrate validator behavior across task families. For example, pair-000001 (medium, pattern "shutdown_configure_restore") shows `normalized_response` identical to the reference and validator fields `parsed_command_count` = 4, `required_command_count` = 4, `observed_touched_field_count` = 3, and `validation_after.valid` = true; pair-000002 (hard, "make_before_break_uplink_switchover") shows `parsed_command_count` = 2 and `validation_after.valid` = true; pair-000004 (very_hard, "safe_access_service_transfer") shows `required_command_count` = 4 and `validation_after.valid` = true. These representative traces are included among the archived execution/scoring artifacts and demonstrate that the validator exercised per-task workflow constraints and recorded concrete parse→state metrics for inspection. Because all episodes were scored as successes (`score` = 1, `scoring_reason_code` = "VALID_CONFIGURATION") the global contingency counts and per-episode validator traces together indicate systematic validator agreement with the generated outputs for this synthetic sample.

Provider tracing, shared generation semantics, and implications. Provider trace fields recorded per-episode `shared_initial_provider_trace_path` entries pointing to `execution/responses/*-shared-initial.txt` and identical `raw_response_sha256` values for baseline and guarded episodes in each pair. These traces, together with `model_calls_used` = 150 and the stage_counts described above, document the operational mechanism that produced identical outputs across conditions when no exclusions were triggered. In short: when detectors produced no high-confidence flags, the guarded pipeline performed no replacements and the shared initial generation was reused for both conditions; this architecture produced perfect concordance (`n11` = 150) and eliminated any generation-level discordance that an independent_generation design could have revealed.

Missing overlap summaries and interpretation. The preregistered fuzzy-overlap detector would have persisted per-item overlap scores and flags if thresholds were crossed. The archive contains no such per-item overlap summary because

the detector implementation persisted overlap outputs only for items that triggered flags or replacements. The absence of persisted overlap scores is therefore an explicit artifact of the detector workflow combined with zero threshold crossings; it prevents reporting of min/median/percentile overlap statistics for this run, and we note this gap as an empirical limitation of the executed logging policy (see `analysis/contamination_summary.csv` and the `analysis_log.jsonl` entry describing detector events).

Synthesis of artifact evidence. The combination of (a) provenance-stable synthetic sampling (`source_identifier` values of the form "synthetic-netops:..." recorded in `execution/task_manifest.jsonl`), (b) shared_initial_candidate generation semantics (`execution/model_configuration.json`), (c) provider call accounting (`analysis/deterministic_reconciliation.json`), and (d) empty `contamination_summary.csv` together explain the degenerate confirmatory outcome: deterministic detectors produced no high-confidence flags and the reuse of a single shared generation per item produced identical baseline and guarded outputs, yielding complete concordance in validator outcomes. All numbers and traces cited above are archived in the execution bundle and referenced by the artifact paths given here (e.g., `execution/raw_results.jsonl`, `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, `analysis/contamination_summary.csv`, `analysis/deterministic_reconciliation.json`).

V. DISCUSSION

Why the result is degenerate: artifact-grounded diagnosis. Two archived execution facts together explain the degenerate null outcome. First, the confirmatory sample was drawn entirely from provenance-stable synthetic tasks generated by `deterministic_netops_task_generator_v5` (task entries carry the "synthetic-netops:" `source_identifier`). Synthetic provenance makes exact normalized token equality and high fuzzy 5-gram overlap with public corpus fingerprints unlikely by construction, and therefore the preregistered high-confidence detector rule (exact match OR 5-gram overlap ≥ 0.80) had little opportunity to trigger. Second, execution semantics set `independent_condition_generation = false` (`shared_initial_candidate`). When the detector produced no high-confidence exclusions, the pipeline reused a single model generation for both baseline and guarded episodes; this reuse is recorded in the model and provider accounting artifacts and reduces independent variance between conditions. Both facts are present in the archived `execution/model_configuration.json` and `execution/task_manifest.jsonl` artifacts and confirmed by the `deterministic_reconciliation` outputs. Together these facts deterministically produce identical baseline/guarded outputs per item and therefore make the paired test uninformative for generation-level contamination effects in this run.

Mechanics of the detector workflow and reporting consequences. The preregistered detector was implemented to persist detailed per-item overlap scores only when threshold

crossings or replacement handling were invoked (to limit storage and to focus archival attention on resolved flags and replacements). Because no items crossed the high-confidence threshold, the detector produced no persisted per-item overlap table in `analysis/contamination_summary.csv`. This implementation choice—record only flagged events—yields a clear archival signal when contamination is detected but produces a blind spot for audits that later wish to inspect the full overlap distribution. The artifact evidence therefore shows an absence of high-confidence flags but does not let us characterize how close items were to the threshold in aggregate. Practically, audit pipelines should persist per-item detector scores at least as summary quantiles so a null flag set can still be interrogated post hoc; we recommend this change precisely because the current archive lacks the overlap distribution needed to rule out near-threshold cases.

Effect on estimand sensitivity and power. The preregistration explicitly contrasted two plausible execution regimes: (A) independent condition generation (two independent model calls per item, one for baseline and one for guarded) and (B) shared generation (single call reused across conditions when exclusions do not apply). Independent generation increases the number of independent model outputs and therefore raises the probability of observing cross-condition discordance arising from generation-level contamination, paraphrase variance, or randomness. The executed shared-generation regime collapses those independent draws into one and therefore reduces observable discordance unless the detector deterministically excludes an item. The execution manifest shows `model_calls_used = 150` versus a planned maximum of 300; this arithmetic difference is the core practical reason the paired estimator had no discordant pairs. The preregistration's power guidance described this tradeoff qualitatively; the artifacts make the tradeoff concrete for this run and explain why the paired test could not detect generation-level leakage under the chosen semantics.

Validator coverage and semantic failure modes. The `deterministic_netops_validator_v5` is a practical, deterministic oracle for the operational properties it encodes: parse→state updates, required command counts, dependency rules, and final-state checks. Validator traces archived per episode (`validation_before/validation_after`) demonstrate that the validator detected and recorded command counts, touched fields, and violation codes for every candidate. However, artifact evidence also makes clear the validator's scope is constrained to the preregistered intent constraints and modeled properties; it does not claim coverage of unmodeled semantic failures (for example, subtle ordering effects outside the specified workflow_policies, vendor CLI idiosyncrasies not represented in the generator, or emergent cross-task interactions). Consequently, a passed validation trace is necessary evidence of meeting the modeled intent but not sufficient evidence of comprehensive production safety. This distinction is present in the archived `validator_trace` fields and motivates our recommendations for richer oracle coverage and human adjudication for borderline or high-risk items.

Concrete reproducibility and artifact inspection guidance. The execution bundle contains the artifacts that substantiate the diagnosis: `execution/model_configuration.json` documents the requested execution semantics (`declared_model_version = "gpt-5-mini"`, `independent_condition_generation` setting), `execution/task_manifest.jsonl` records each sampled task with `source_identifier` and `generation_seed`, `analysis/contamination_summary.csv` is empty (no high-confidence flags), `analysis/paired_contingency_table.csv` records `n11 = 150` and zero discordant pairs, and `analysis/paired_difference_figure.svg` contains the bootstrap distribution used to compute the CI. Inspecting these artifacts in the archive allows an auditor to verify the exact pipeline behavior without recomputing model outputs. We highlight that persistent per-item detector scores were intentionally omitted when flags were absent; auditors who require those scores should request pipelines that persist per-item summaries regardless of flag status.

Design prescriptions to increase audit sensitivity. Artifact-grounded lessons suggest specific, implementable changes: (1) sample provenance-rich, web-sourced items (with stable URLs/DOIs and timestamps) to exercise the fuzzy-overlap and provenance heuristics; (2) set `independent_condition_generation = true` and execute two independent model calls per item to create the independent comparisons that the paired test is designed to detect; (3) increase model diversity (multi-model replication) to surface model-specific memorization patterns; (4) persist per-item fuzzy-overlap scores and canonicalized fingerprints even when thresholds are not crossed so the overlap distribution is reconstructible; and (5) add paraphrase-robust detectors (semantic similarity beyond `n`-gram overlap) plus human adjudication for borderline results. Each prescription follows directly from the execution artifacts: the `model_calls_used` shortfall, the empty `contamination_summary`, and the recorded `shared_initial_candidate` semantics.

Operational implications and auditor tradeoffs. Auditors face tradeoffs between reproducibility and sensitivity. Deterministic synthetic sampling and shared generation maximize reproducibility and reduce compute, but those same choices can eliminate the observable signal needed to detect training-data leakage at generation time. Conversely, independent generations and multi-model designs increase sensitivity but require more calls, storage, and non-determinism management. The archived execution demonstrates that reproducibility-focused choices can produce a valid confirmatory test that nevertheless answers a narrower methodological question than originally targeted; explicit preregistration of those tradeoffs (as in our archive) is therefore essential for correct interpretation.

Broader methodological takeaways. Artifact-anchored preregistration remains a powerful practice: freezing detectors, thresholds, replacement rules, and analysis executors before observing outcomes prevents post hoc rule changes. However, the audit shows that preregistration must also commit to sampling and generation semantics that align with the

estimand’s sensitivity requirements. In short, auditors should preregister not only detectors and thresholds but also the independence structure of model draws and the persistence policy for detector outputs; the execution artifacts here show all three choices materially affected inferential ability.

VI. LIMITATIONS

- Single-model execution (`gpt-5-mini`) — results do not generalize across other LLM families, sizes, or training corpora.
- Sampled items were provenance-stable synthetic/deterministically generated tasks (`source_identifier` prefix `"synthetic-netops:"`) for this run; absence of high-confidence flags here may not reflect contamination prevalence in real-world, web-crawled benchmarks.
- Generation semantics used `shared_initial_candidate` across baseline and guarded conditions (`independent_condition_generation = false`), producing identical model outputs when no exclusions occurred and thus limiting sensitivity to interventions that rely on independent re-generation.
- Contamination detection relied on preregistered conservative heuristics (exact match and fuzzy 5-gram overlap thresholds); subtler contamination patterns (paraphrased memorization, partial leakage) may escape these heuristics and require richer provenance or human adjudication.
- Passing the `deterministic_netops_validator_v5` indicates satisfaction of the checked functional constraints but does not imply comprehensive production readiness or absence of semantic regressions outside the validator’s scope.

VII. CONCLUSION

We executed a fully preregistered, artifact-anchored paired audit of conservative contamination detectors for LLM-for-NetOps evaluation. In the reproducible 150-pair synthetic sample we observed zero preregistered high-confidence contamination flags and identical validator pass rates (150/150 baseline, 150/150 guarded). The degenerate null outcome is artifact-supported and stems from provenance-stable synthetic sampling and shared generation semantics; it does not imply absence of contamination in web-sourced benchmarks or failure of the preregistered detectors in different sampling regimes. We provide an explicit preregistration→execution reconciliation, confirm deterministic reconciliation checks passed, and recommend concrete design changes—provenance-rich sampling, independent generation per condition, multi-model scope, paraphrase-sensitive detectors, and matched power calculations—to increase audit sensitivity in future studies. All experiment artifacts, validator traces, analysis inputs/outputs, and execution manifests are archived and available as recorded in the Disclosure Statement below.

DISCLOSURE STATEMENT

This study was executed under the autonomous-run preregistration contamination-audit-netops-2026-v1 (preregistration

SHA-256 recorded in the archived bundle). LLM used: gpt-5-mini (declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic task generator: deterministic_netops_task_generator_v5 (version 5.0). Deterministic validator: deterministic_netops_validator_v5 (version 5.0). Representative executed artifacts (by path) include execution/model_configuration.json, execution/task_manifest.jsonl, execution/raw_results.jsonl, analysis/paired_contingency_table.csv, and analysis/contamination_summary.csv; the full execution bundle contains raw responses, validator traces, execution logs, and the transformation manifest. Exact immutable initial master-prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Topic constraints (Generative AI and LLMs for NetOps) were selected by the human Research Director prior to the autonomy lock. After the autonomy lock, the AI system autonomously performed literature search, gap selection, preregistration, experiment execution, analysis, manuscript drafting, peer-review simulation, and revision. All generation prompts, model outputs, validator traces, sampling seeds, replacement rules, execution logs, and analysis artifacts were produced and archived by the autonomous pipeline and are included in the execution bundle. No manual human editing of technical content, analyses, references, figures, tables, captions, or the Disclosure Statement occurred after the autonomy lock. Pre-lock development and rehearsal runs occurred (permitted) but pre-lock artifacts were not used as empirical evidence for this final study unless re-created under the locked pipeline. The execution followed preregistered missingness, retry, and exclusion policies; no deviations occurred.

REFERENCES

- [1] <https://arxiv.org/abs/2604.22513>
- [2] <https://doi.org/10.48550/arxiv.2606.06212>
- [3] <https://doi.org/10.48550/arxiv.2605.22092>
- [4] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [5] <https://doi.org/10.1007/s11704-026-60308-3>
- [6] Buğra Alperen Uluirmak, Rifat Kurban, "EvalSafetyGap: A Hybrid Survey and Conceptual Framework for LLM Evaluation-Safety Failures", 2026
- [7] Nguyen Van Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, Ting Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", 2024, 10.1145/3703155